Masterplan: AI-Powered Parental Task Delegation App

(Working title — rename freely)

---

1. App Overview & Objectives

This app allows a child who has **no independent internet access** (a basic keypad phone, or a smartphone with no data) to get everyday tasks done — like ordering food or booking a cab — simply by making a normal phone call to their parent.

The parent's phone runs a companion app that, once the call ends, transcribes the conversation, figures out what the child needs, prepares the action, and shows the parent a short, tappable summary. The parent approves or declines with a single tap, and only then is the task actually carried out.

The core objective: reduce the mental and logistical load on a busy parent. Instead of stopping an important task to personally open an app and place an order for their child, the parent gets a pre-packaged decision — approve or decline — and the system handles the rest.

---

2. Target Audience

- **Primary users (app owners):** Parents/guardians of children roughly **ages 5–14**, particularly those whose children don't yet have independent smartphone/data access.
- **Indirect users (never touch the app):** The children themselves — they only ever interact via a normal phone call, regardless of whether they have a basic keypad phone or a smartphone.
- **Household structure:** Designed from the start to support **multiple children per parent account**, each with their own profile, phone number, and individually tuned settings.

(Note: the long-term vision includes expanding beyond children — e.g., elderly parents or others without digital independence — but v1 is deliberately scoped to the child use case.)

---

3. Core Features & Functionality

3.1 Voice Call Intake

- Child calls the parent's registered number as a completely normal phone call.
- The app identifies the incoming call, verifies it against a registered child profile, and passively processes it — no changes to how the child places the call at all.

3.2 Post-Call Processing (Batch, not real-time)

- Once the call ends, the recording is transcribed to text.
- A decision-making AI layer interprets intent and identifies whether an actionable task is present.

3.3 Task Categories (v1 scope)

1. **Food ordering**
2. **Cab/bike booking**
3. **Emergency handling** — bypasses the normal flow entirely (see 3.5)

3.4 Approval Flow — "Prepare → Approve → Execute"

- The AI fully prepares the task (e.g., restaurant, items, cost, delivery address) but does **not** take real-world action yet.
- The parent sees a short, plain-language summary card with **Accept** / **Decline** buttons.
- The actual task is only executed after the parent taps **Accept**. Nothing irreversible happens before approval.
- If declined (or no response, for non-emergency tasks), the task is discarded — no action taken.

3.5 Emergency Handling (Hard Bypass)

- If genuine distress is detected in the call, this **skips the normal approval pipeline completely**.
- Immediately: parent is urgently notified, the child's location is auto-shared, and a backup contact is alerted if the parent doesn't respond in time.

3.6 Multi-Child Support

- Each child has an individual profile: registered phone number, age, and individually adjustable approval strictness.

- Age acts as a *starting default* for how cautious the system is (younger children default to "approval required for nearly everything"; older children can have low-risk/routine actions handled with lighter oversight) — but this is a tunable setting per child, not a hardcoded rule.

3.7 App Structure (Parent-Facing)

- **Home** — manage registered child contacts/profiles.
- **History** — log of all past tasks (transcEl script summary + outcome: approved/declined/executed), no raw audio retained.
- **Profile** — parent account details.

---

4. High-Level Technical Stack Recommendations

(Conceptual only — no implementation specifics)

Layer	Recommendation	Why
Mobile app (parent-facing)	Native Android, built in **Java**	Matches your existing skillset directly; Android also has strong market share in your likely target region, so starting Android-only doesn't meaningfully limit reach for v1.
AI / decision-making backend	A separate **Python-based service**	This is where speech-to-text, intent understanding, and task preparation logic live — plays directly to your ML/Python strengths. Keeping it separate from the mobile app means the two can evolve independently.
Task execution layer ("agent")	Simulated/mock execution for v1	Real platforms (Zomato, Swiggy, Uber, Ola) generally don't offer public APIs for third-party order placement. Simulating this lets you prove the hardest, most novel part of the product — the voice → decision → approval loop — without depending on external partnerships you can't control yet.
Database	A lightweight, free-tier-friendly managed database	You're only storing transcripts, summaries, and task outcomes — a small, well-structured dataset that doesn't need heavy infrastructure to start.
Notifications	Push notifications	Needed to alert the parent the moment a task summary is ready, or immediately for emergencies.

5. Conceptual Data Model

- **Parent (Account)**
  - Contact info, login credentials, linked child profiles
- **Child Profile**
  - Linked to one parent
  - Registered phone number
  - Age → default approval-strictness setting (adjustable)
  - Voice biometric enrollment sample
  - Backup emergency contact
- **Task Record**
  - Linked to a child profile
  - Call transcript + AI-generated summary
  - Task category (food / cab / emergency)
  - Status: pending / approved / declined / executed / failed
  - Timestamp
  - *(No raw audio stored beyond in-the-moment processing)*
- **Emergency Event**
  - Linked to a child profile
  - Trigger timestamp, location snapshot, backup contact escalation status

---

6. User Interface Design Principles

- **Minimal friction, maximum clarity.** The parent should never need more than a glance and one tap to make a decision.
- **Flow after a call:** Call ends → brief loading state → short plain-language summary card → Accept/Decline.
- **Three-section structure:** Home (manage children) / History (past tasks) / Profile (account).
- **Emergency UI is visually distinct** from routine task cards — it should never be mistakable for a normal approval request, and should not require the same "tap to proceed" pattern (parent should be alerted, not asked).

---

7. Security Considerations

- **Caller verification, two layers:**
  1. Incoming call number must match the child's registered number.
  2. **Voice biometric matching** confirms it's genuinely the enrolled child speaking (requires a one-time voice enrollment step per child during setup).
- **Data minimization:** only transcripts, summaries, and task outcomes are stored — no long-term raw audio retention.
- **Execution is approval-gated** (Flow 2) — no real-world, irreversible action occurs without explicit parent consent, except the emergency bypass, which is intentionally designed to skip consent in favor of speed and safety.
- **Emergency escalation path** ensures a non-responsive parent doesn't become a single point of failure during a genuine crisis.

---

8. Development Phases / Milestones

Phase 1 — Core Loop (MVP)

- Android app with Home/History/Profile
- Call detection + number verification (voice biometrics can start simple, refined later)
- Backend: transcription + decision engine
- Simulated execution for food & cab tasks
- Approve/decline flow (Flow 2)
- Basic history logging

Phase 2 — Safety & Trust Hardening

- Full voice biometric enrollment/verification
- Emergency bypass flow: location sharing + backup contact escalation
- Per-child, age-based approval-strictness settings

Phase 3 — Real-World Execution

- Explore real integration paths for food/cab platforms (partnership outreach and/or automation approach)
- Move from simulated to (partially) real task execution

Phase 4 — Expansion

- iOS / cross-platform support

- Broader audience beyond children (elderly, low-connectivity users generally)
- Additional task categories
- Optional web dashboard for parents

---

9. Potential Challenges & Solutions

Challenge	Solution / Mitigation
No public APIs for food/cab platforms	Start with simulated execution for MVP; pursue partnerships or automation approaches once the core product is validated
False positives/negatives in emergency detection	Keep the bypass trigger conservative initially; log and review transcripts to refine detection over time
Children's voices change as they grow, affecting biometric accuracy	Periodic re-enrollment prompts as the child ages
Parent doesn't respond to an approval request in time	Defined timeout behavior: routine tasks default to "no action taken"; emergencies escalate to backup contact instead of stalling
Reliability of call detection across carriers/devices	Early technical validation needed on how call audio access works across different Android versions and carriers

---

10. Future Expansion Possibilities

- Broaden beyond children to other dependents (elderly parents, low-literacy users, temporarily disconnected users)
- iOS support / full cross-platform coverage
- Real, sanctioned integrations with major food delivery and ride-hailing platforms
- Expanded task categories (e.g., scheduling, reminders, information lookups)
- Optional passphrase or enhanced biometric security tiers
- Web-based dashboard for parents to review settings/history from a desktop

---

This document reflects the conceptual blueprint as of initial planning. It's meant to evolve — revisit and update it as the product takes shape.